

TP n°4 – Découverte de OpenTofu

Table des matières

- 1. Une VM avec vagrant pour openstack et OpenTofu
 - 1.1. Machine virtuelle
 - 1.2. Configuration de l'authentification openstack
- 2. Premier pas avec OpenTofu
 - 2.1. Choix et déclaration du provider
 - 2.2. Une première instance
- 3. Information sur l'infrastructure déployée, les variables de sortie
- 4. Utilisation de cloud-init avec tofu
- 5. Utilisation d'une image personnalisée
- 6. Pour aller plus loin
- 7. Encore plus loin, description de l'infrastructure dans une variable d'entrée
- 8. Déploiement d'application avec modules et docker/podman

1. Une VM avec vagrant pour openstack et OpenTofu

1.1. Machine virtuelle

Vous trouverez sur moodle un `Vagrantfile` permettant de créer une machine virtuelle provisionnée avec les outils nécessaires à la réalisation de ce TP.

En particulier, y seront installés les commandes :

- OpenTofu
- openstack

La commande `openstack` permet d'interroger un environnement OpenStack en ligne de commande et nous permettra de vérifier que votre environnement est configuré correctement

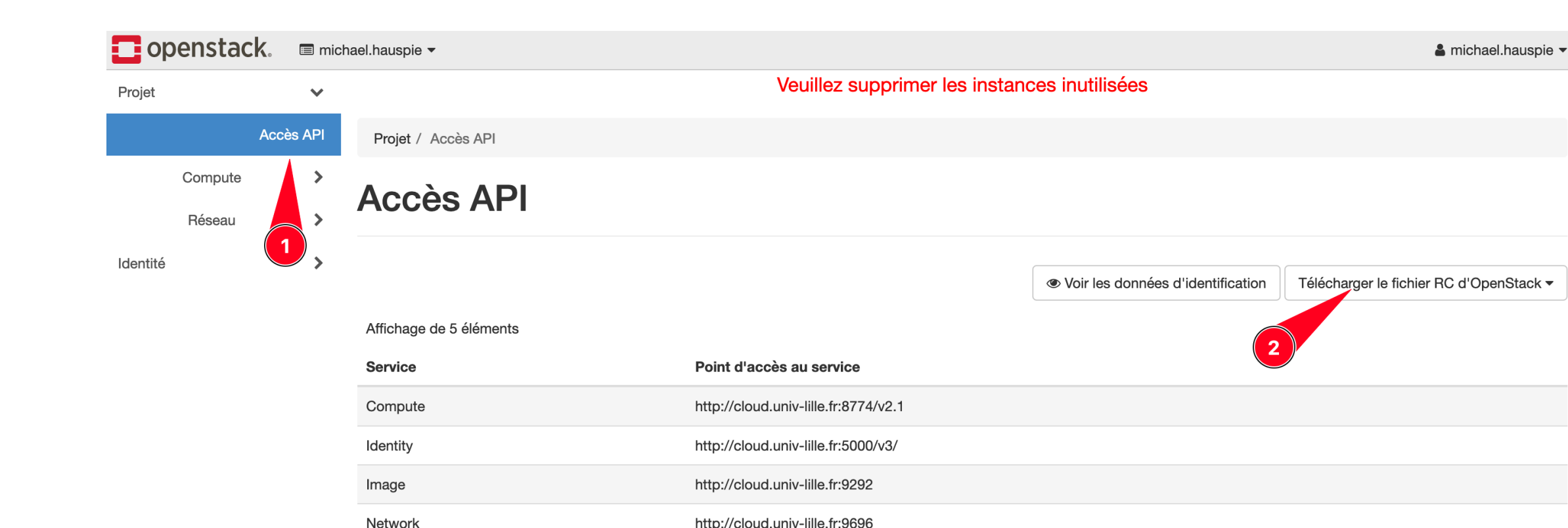
- Télécharger le fichier `Vagrantfile` sur moodle
- Démarrer la machine à l'aide de la commande `vagrant up`

1.2. Configuration de l'authentification openstack

Pour s'authentifier sur openstack, et pouvoir utiliser OpenTofu pour lancer des instances, il faut affecter certaines variables d'environnement (login, mot de passe, url pour l'authentification...)

Ces variables sont nombreuses et dépendent de la configuration de l'environnement Openstack que vous utilisez. Heureusement, vous pouvez télécharger un script qui positionnera les variables nécessaires dans votre shell.

- Se connecter sur l'Openstack de l'université : <https://cloud.univ-lille.fr>
- Dans le menu *Accès API* (1) choisir *Télécharger le fichier RC d'OpenStack* (2), plus précisément *Fichier OpenStack RC (Identity API v3)*



- Placer ce fichier (qui par défaut devrait porter un nom de la forme `login-openrc.sh`) dans le répertoire qui contient votre `Vagrantfile`
- Dans la VM, se placer dans le répertoire `/vagrant` et charger le fichier précédent dans l'environnement du shell courant à l'aide de la commande `source`. Saisir votre mot de passe lorsqu'il vous est demandé. **Attention** votre mot de passe sera stocké **en clair** dans la variable d'environnement `OS_PASSWORD`. Le contenu de cette variable n'est accessible que du processus qui exécute votre shell et de ses processus fils. Attention à ne pas l'afficher (par un `echo` ou un appel à la commande `env`)
- Tester la connexion à OpenStack avec la commande `openstack flavor list`. Cette commande liste les gabarits d'instances disponibles pour la création d'instances. Le module *provider* OpenTofu pour OpenStack utilise les mêmes variables d'environnement que la commande `openstack` pour sa configuration.

Vous pouvez continuer le TP s'il n'y a pas d'erreur lors de l'exécution de la commande `openstack` et que la sortie est similaire à :

ID	Name	RAM	Disk
17c669b5-8b3d-4e9a-a396-b71d877326df	tres puissante avec stockage++	8192	60
19179fcb-add4-4210-9a28-45f0123136cc	tres puissante	8192	40
460b3ffb-5f01-4d95-a3d1-7c7ec60f4e56	puissante	4096	20
4ac42f52-53ad-47d0-ac44-9dd9a9f8ea99	moyenne	2048	10
689721ab-4479-4406-b133-ab9db72b7910	normale	1024	10
69a2a928-e0f3-4fb3-8de6-b21eb8f30620	petite	512	5

Dans le cas contraire, reprendre les étapes précédentes en vérifiant bien chacune d'elles.

2. Premier pas avec OpenTofu

2.1. Choix et déclaration du provider

La première étape pour travailler avec OpenTofu est de choisir un fournisseur cloud. Dans notre cas, il s'agit d'OpenStack. Il faut donc utiliser le module *provider* OpenTofu compatible avec OpenStack.

- À l'aide du [registre OpenTofu](#), trouver le *provider* nécessaire pour OpenStack
- Créer un fichier `main.tf` (toujours dans le même répertoire), et y saisir les déclaration nécessaires pour utiliser le *provider* OpenStack. Comme les variables d'environnements nécessaires ont été correctement configurées dans la section précédente, nous n'avons besoin que de la section située dans

```
terraform {
  required_providers {
    /*...*/
  }
}
```

- Lancer la commande `tofu init` pour télécharger le module *provider* et préparer l'environnement tofu

2.2. Une première instance

Nous allons maintenant lancer une première instance sur OpenStack à l'aide de tofu.

Dans un premier temps, assurez vous de ne pas avoir d'instance de machine virtuelle en fonctionnement sur votre OpenStack (à l'aide de l'interface web ou, mieux, de la commande `openstack`). Si c'est le cas, supprimez les. Nous ne disposons que d'un quota très limité sur l'OpenStack.

Les objets dans tofu se gèrent grâce à des *ressources*. Ces ressources sont déclarées dans votre fichier `main.tf` sous la forme :

```
resource "type_de_ressource" "nom_de_ressource" {
  /* configuration de la ressource */
}
```

- "type_de_ressource" dépend du *provider* et du type d'objet que l'on veut gérer (instance de VM, réseau, image de disque...)
- "nom_de_ressource" est simplement le nom que vous donnez à la ressource, il peut être choisi librement et pourra parfois être utilisé à d'autres endroits dans le fichier pour y faire référence

Pour créer une instance sur OpenStack, le type de ressource qui nous intéresse est `"openstack_compute_instance_v2"`

- Lire la documentation du module *provider* openstack, en particulier la section *Compute / Nova > Resources > openstack_compute_instance_v2*
- Quelle sont les variables à renseigner *obligatoirement* pour pouvoir créer une ressource de type `openstack_compute_instance_v2`. Indice, il y en a trois.
- Ajouter une ressource à votre fichier permettant de créer une instance nommée `vm1` (on utilisera également `vm1` comme nom pour la ressource), à partir de l'image `debian13` et utilisant le gabarit `petite`
- Lancer la commande `tofu plan`. Cette commande indique ce que Tofu devrait faire pour respecter vos souhaits. Il indique les ressources à créer, à supprimer, à modifier...
- Si tout semble correspondre, lancer la commande `tofu apply` pour appliquer le plan de déploiement
- Vérifier (sur l'interface web ou à l'aide de la commande `openstack`) que votre machine virtuelle a bien été créée

3. Information sur l'infrastructure déployée, les variables de sortie

Tofu peut, après l'application du plan de déploiement, nous donner des informations sur celui ci grâce à des variables de sortie.

Celles-ci se présentent sous la forme :

```
output "variable" {
  value = expression
}
```

où "variable" est un nom que l'on choisi librement et `expression` est une expression faisant référence à des objets liés à l'infrastructure. Les objets *ressources* sont référencé à l'aide d'une expression de la forme `type_de_ressource.nom_de_ressource`. Ainsi, l'objet correspondant à l'instance que nous avons déployée dans l'exercice précédent s'écrit `openstack_compute_instance_v2.vm1`.

Pour chaque type d'objet, on peut alors accéder à différentes variables qu'on l'on peut trouver dans la documentation de chaque type.

- Quelle variable permet de connaître l'adresse IPv4 d'une instance OpenStack ?
- Écrire les directives nécessaires dans le fichier `main.tf` pour définir une variable de sortie nommée `"ip_vm1"` dont la valeur est l'adresse IPv4 de l'instance `vm1`
- Lancer la commande `tofu output`, que se passe-t'il ?
- Appliquer la correction suggérée par tofu et vérifier que la commande `tofu output` fonctionne correctement et que vous obtenez bien l'adresse de votre instance

4. Utilisation de cloud-init avec tofu

Nous allons maintenant combiner cloud-init et tofu.

- Trouver dans la documentation de la ressource `openstack_compute_instance_v2`, le nom de la variable à utiliser pour fournir les *user data* à l'instance
- Utiliser cette variable pour que votre clé publique SSH soit déployée sur la VM (c.f. TP précédent). Pour donner le contenu de la variable, vous pouvez le faire directement sous forme de chaîne de caractères (attention, il faut indiquer les saut de lignes avec `"\n"`), mais c'est peu pratique, ou indiquer à tofu de charger le contenu d'un fichier avec la fonction [file](#)

À la fin de cette partie, supprimer l'infrastructure avec `tofu destroy`

5. Utilisation d'une image personnalisée

On souhaite utiliser notre propre image au lieu de l'image `debian13` fournie par l'université.

Trouver, dans la documentatio du module openstack, comment écrire votre fichier tofu de façon à utiliser l'image <https://cloud.debian.org/images/cloud/bookworm/latest/debian-12-genericcloud-amd64-qcow2> pour votre machine virtuelle.

Vous ne devez pas télécharger vous même cette image, uniquement écrire les bonnes directives dans votre fichier tofu.

6. Pour aller plus loin

On souhaite maintenant déployer, non pas une, mais trois instances

- Modifier votre fichier `main.tf` pour créer trois ressources, nommées `vm0`, `vm1`, `vm2` et tester le déploiement (ne pas oublier d'ajouter des variables de sorties pour l'ip de chaque nouvelle instance)
- Lire la documentation sur le [méta-argument count](#) et modifier le fichier de façon à effectuer le même déploiement, mais en écrivant la ressource qu'une seule fois. Trouver une solution générique (c'est à dire qui marche, quelque soit le nombre d'instances créées) pour l'affectation des adresses IP dans les variables de sorties (un indice, une solution utilise ce qui est présenté [ici](#))

7. Encore plus loin, description de l'infrastructure dans une variable d'entrée

Cette fois, on souhaite décrire notre infrastructure dans une variable d'entrée. Celle ci se présente sous la forme :

```
variable "projet" {
  description = "La configuration de notre infra"
  type = map
  # Chaque élément de notre map est une VM qui utilise un gabarit
  # différent
  default = {
    vm1 = {
      flavor = "normale",
      hostname = "vm1",
    },
    vm2 = {
      flavor = "petite"
      hostname = "vm2",
    },
    vm3 = {
      flavor = "petite",
      hostname = "vm3",
    },
  }
}
```

Cette directive définit une variable "projet" qui est une *map* (un dictionnaire), dont la valeur par défaut est telle qu'indiquée. On va donc s'en servir pour créer une infrastructure en fonction de cette valeur. Donc ici, on souhaite créer trois instances, nommée `vm1`, `vm2` et `vm3` qui utilisent des gabarits différents. On ne pourrait donc pas le réaliser facilement avec un `count`.

- Lire la documentation sur le [méta-argument for_each](#) et modifier votre fichier pour créer l'infrastructure en fonction de la variable "projet". Cette variable peut être accédée dans la ressource par `var.projet`.

8. Déploiement d'application avec modules et docker/podman

Écrire un module opentofu qui permet de déployer complètement une application conteneurisée sous docker/podman à partir de paramètres d'entrées.

Votre module doit attendre en entrées les paramètres suivants:

- nom de l'image docker/podman de l'application à déployer ;
- port à exposer sur la VM qui fera tourner docker/podman ;
- liste de variables d'environnements et leur valeurs qui seront passées au conteneur lors du déploiement de l'application.

Votre module terraform doit comprendre toutes les ressources et manipulation nécessaires pour

- créer une VM
- y déployer une clé SSH afin de pouvoir s'y connecter en tant que root
- s'assurer que la VM est complètement à jour
- installer docker ou podman sur cette VM
- déployer l'application en lui passant les bon paramètres

Votre module fournira alors en variable de sortie :

- l'adresse IP de la VM
- le port sur lequel est accessible le service